

24CSA212 – Full Stack Frameworks

INTRODUCTION TO NODE.JS AND NPM

Node.js Modules

- Modules in Node.js provide functionality that can be used throughout the Node.js application
- Node.js has many modules to provide required functionality for a web application

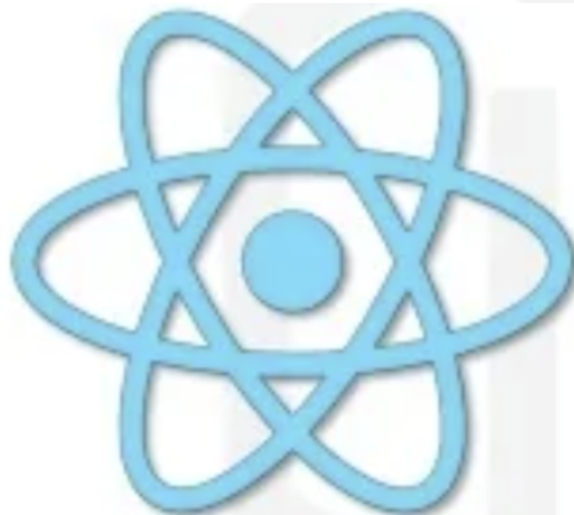


Node.js Modules

Some of the popular modules are:

- express
- react
- Nodemon

Express **JS**



What is NPM?

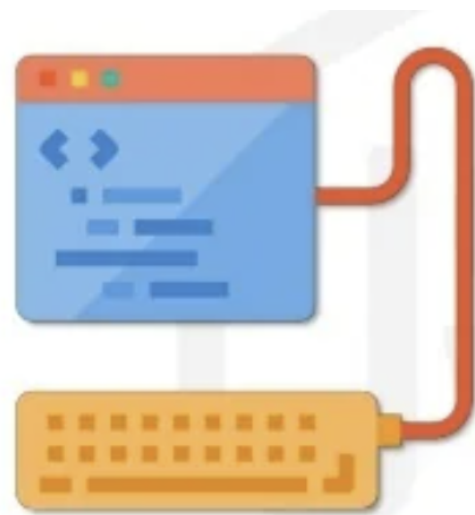
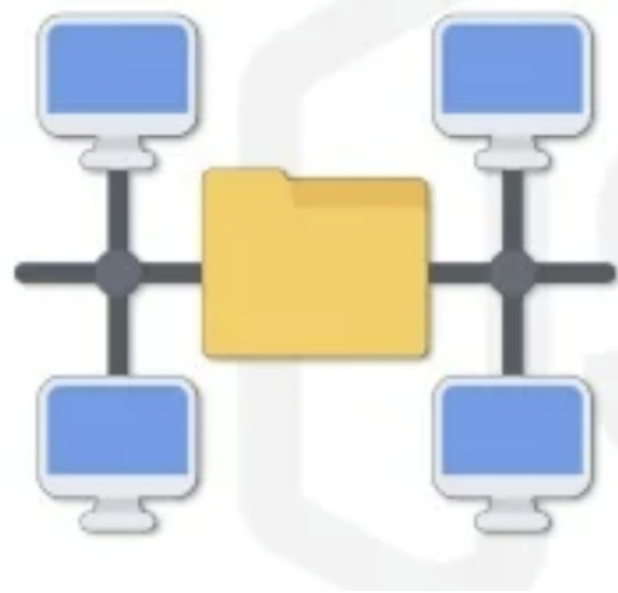
- NPM stands for Node Package Manager
- NPM is responsible for managing all the packages and modules for Node.js



What is NPM?

Node Package Manager provides two main functionalities:

- It provides online repositories for node.js packages/modules which are searchable on search.nodejs.org
- It also provides command line utility to install Node.js packages, does version management and dependency management of Node.js packages



Why NPM?

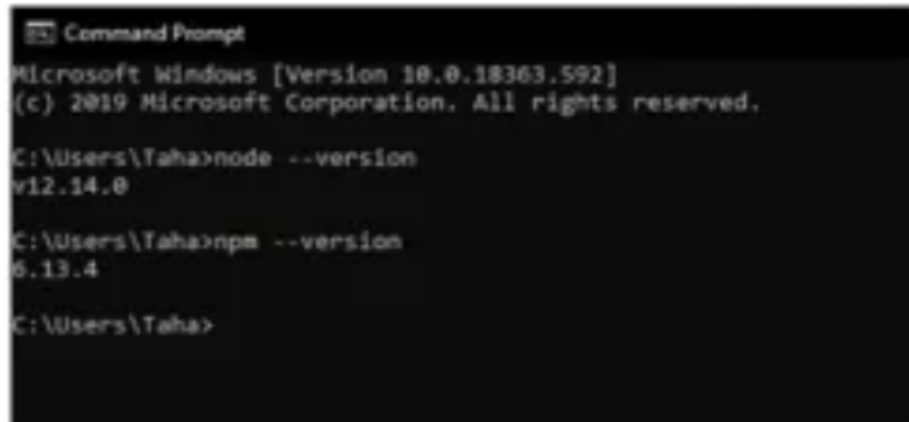
- Helps in adding the required packages into a project
- Manages and maintains various versions of codes and their dependencies
- NPM automatically updates the application when the project code undergoes any change

NPM Installation

- Node Package Manager is included with the installation of Node.js

NPM Installation

- Download Node.js from <https://nodejs.org/en/download/>
- Run the downloaded Node.js installer and accept the default settings
- Verify if Node.js is properly installed using command prompt: `node --version`
- NPM gets installed alongside, verify if it is properly installed using command prompt: `npm --version`



```
Command Prompt
Microsoft Windows [Version 10.0.18363.592]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\Taha>node --version
v12.14.0

C:\Users\Taha>npm --version
6.13.4

C:\Users\Taha>
```

INSTALLING THIRD PARTY MODULES

➤ NPM Install

npm install <options> <package unique name>

Eg - npm install express --save

NODE.JS “REQUIRE”

The modules/packages available can be imported in .js file using “require” function.

Ex – `require(“http”);`

This helps in loading “http” package in current .js file.

MODULES EXPORT N IMPORT

- A module in node.js can be exported, by which it can be imported in other files.
- The “require” function helps in importing modules.
- Exporting -
`module.exports{<module to be exported>}`
- Importing –
`var <varName> = require(<module name>);`

Package.json File

- The **package.json** file is the heart of a Node.js system
- package.json holds the metadata about a particular project
- package.json is present in the root directory of any Node application or module
- package.json provides the Node package manager with the necessary information to understand how the project should be handled along with its dependencies.



Package.json File

```
{
  "name": "node-npm",
  "version": "1.0.0",
  "description": "A demo application",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Taha",
  "license": "ISC"
}
```

package.json

- package.json file after creating a Node.js project using the command: `npm init`
- You can edit these parameters at the time of creation of a Node.js project

Explanation:name: The name of the application/project.

- version:** The version of application. The version should follow semantic versioning rules.
-

- description:** The description about the application, purpose of the application, technology used like React, MongoDB, etc.
- main:** This is the entry/starting point of the app. It specifies the main file of the application that triggers when the application starts. Application can be started using **npm start**.
- scripts:** The scripts which needs to be included in the application to run properly.
- engines:** The versions of the **node** and **npm** used. These versions are specified in case the application is deployed on cloud like heroku or google-cloud.
- keywords:** It specifies the array of strings that characterizes the application.
- author:** It consist of the information about the author like name, email and other author related information.
- license:** The license to which the application confirms are mentioned in this key-value pair.

- **dependencies:** The third party package or modules installed using.
- **npm:** are specified in this segment.
- **devDependencies:** The dependencies that are used only in the development part of the application are specified in this segment. These dependencies do not get rolled out when the application is in production stage.
- **repository:** It contain the information about the type and url of the repository where the code of the application lives is mentioned here in this segment.
- **bugs:** The url and email where the bugs in the application should be reported are mentioned in this segment.

Note: Here, “**body-parser**”, “**express**”, “**express-validator**”, “**mongoose**” and “**nodemon**” are the modules/packages installed using **npm (Node Package Manager)**.

Getting started & hello world

- Install the latest LTS version of Node.js from <https://nodejs.org/en/>
- Open an editor of your choice and start typing JavaScript.
- Save the file with a '.js' extension.
- Run it by using the following command

```
node filename.js
```


Node.js First Application

Console based Node.js Application: The Node.js console-based applications are run using Node.js command prompt.

The `console.log()` function is used to display output on console. This function prints output to stdout with newline.

```
console.log('Hello World');
```

Web-based Node.js Application: A web-based Node.js application consists of the following three important components:

Import required modules: Load Node.js modules using the require directive. Load http module and store returned HTTP instance into a variable.

Syntax: `var http = require("http");`

Create server: Create a server to listen the client's requests. Create server instance using `createServer()` method. Bind server to port 8080 using `listen` method associated with server instance.

Syntax: `http.createServer().listen(8080);`

Read request and return response: Read the client request made using browser or console and return the response. A function with request and response parameters is used to read client request and return response.

Syntax:

`http.createServer(function (request, response) {...}).listen(8080);`

Run the file on Node.js command prompt using command **node firstprogram.js** and type ***http://127.0.0.1:8080/*** in a web browser to see the output.

Example 2 - HTTP Server

Following code creates an HTTP Server and prints 'Hello World' on the browser.



```
const http = require('http');

const server = http.createServer(function (req, res) {
  res.writeHead(200);
  res.end('Hello, World!');
});

server.listen(8080);
```

HELLO WORLD

```
var http = require('http');  
http.createServer(function (req, res) {  
  console.log('Receiving Request');  
  res.writeHead(200, {'Content-Type': 'text/plain'});  
  res.write('Hello World\n');  
  res.end();  
}).listen(8081, 'localhost');  
  
console.log('Server running at http://localhost:8081/');
```

Example 3 - TCP Server

Here is an example of a simple TCP server which listens on port 3000 and echoes whatever you send it.



```
const net = require('net');

net.createServer((socket) => {
  socket.write('Server\n');
  socket.pipe(socket);
}).listen(3000, '127.0.0.1');
```

ASYNCR/O

```
var fs = require('fs');

console.time('Start timer Read');
console.time('Start timer Write');

fs.readFile('test.txt', function(err, data) {
  if (err) {
    console.log("File Reading err: " + err);
    return;
  }
  console.log("File Reading: " + data );|
  console.timeEnd('Start timer Read');
});

fs.writeFile('write.txt', 'Writing to file 111 3333', function() {
  console.log("Done with file write \n");
  console.timeEnd('Start timer Write');
});

console.log('Done \n');
```


SYNC I/O

```
var fs = require('fs');

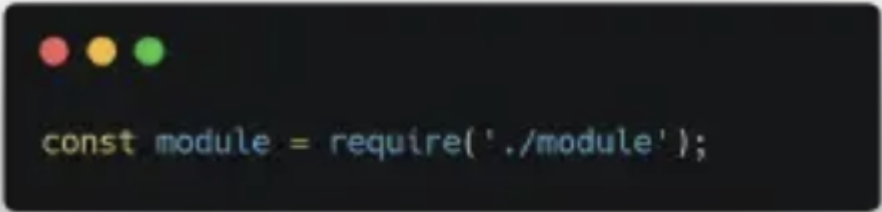
console.time('Start timer');

var data = fs.readFileSync('test.txt');
console.log("File Read: " + data);
console.timeEnd('Start timer');

fs.writeFileSync('write.txt', 'Writing to file 111 3333');
console.log("Done with file write");
console.timeEnd('Start timer');
console.log('Done');
```

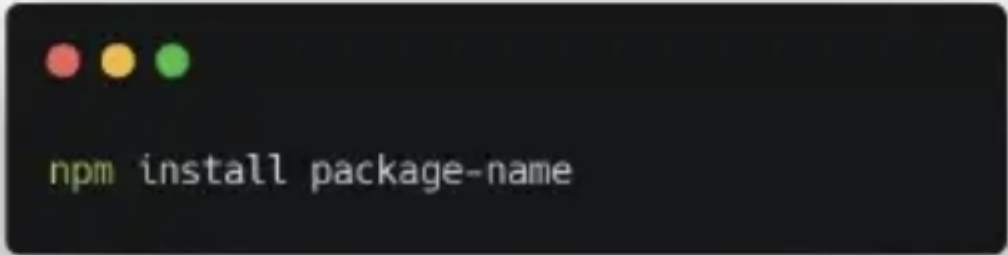
Node.js Ecosystem

- Node.js heavily relies on modules, in previous examples require keyword loaded the http & net modules.
- Creating a module is easy, just put your JavaScript code in a separate js file and include it in your code by using keyword require, like:



```
const module = require('./module');
```

- Libraries in Node.js are called packages and they can be installed by typing
 - NPM (Node Package Manager) comes bundled with Node.js installation.
-

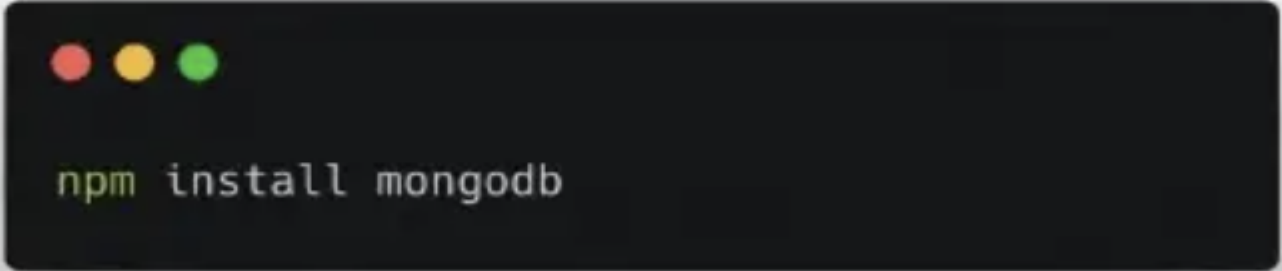


```
npm install package-name
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text `npm install package-name` is displayed in a light green monospace font.

Example 4 - Connect to MongoDB

Install MongoDB using npm, the official MongoDB driver for Node.js.



```
npm install mongodb
```

Datatypes:

Node.js contains various types of data types similar to JavaScript.

- Boolean
 - Undefined
 - Null
 - String
 - Number
-

Loose Typing: Node.js supports loose typing, which means you don't need to specify what type of information will be stored in a variable in advance. We use the var and let keywords in Node.js declare any type of variable.

```
// Variable store number data type
let a = 35;
console.log(typeof a);

// Variable store string data type
a = "GeeksforGeeks";
console.log(typeof a);

// Variable store Boolean data type
a = true;
console.log(typeof a);

// Variable store undefined (no value) data type
a = undefined;
console.log(typeof a);
```

Objects & Functions: Node.js objects are the same as JavaScript objects i.e. the objects are similar to variables and it contains many values which are written as name: value pairs. Name and value are separated by a colon and every pair is separated by a comma.

```
let company = {  
  Name: "GeeksforGeeks",  
  Address: "Noida",  
  Contact: "+919876543210",  
  Email: "abc@geeksforgeeks.org"  
};  
  
// Display the object information  
console.log("Information of variable company:", company);  
  
// Display the type of variable  
console.log("Type of variable company:", typeof company);
```


Output:

Information of variable company: {

Name: 'GeeksforGeeks',

Address: 'Noida',

Contact: '+919876543210',

Email: 'abc@geeksforgeeks.org'

}

Type of variable company: object

Functions: Node.js functions are defined using the **function** keyword and then the name of the function and parameters which are passed in the function. In Node.js, we don't have to specify datatypes for the parameters and check the number of arguments received. Node.js functions follow every rule which is there while writing JavaScript functions.

```
function multiply(num1, num2) {  
    // It returns the multiplication  
    // of num1 and num2  
    return num1 * num2;  
}  
  
// Declare variable  
let x = 2;  
let y = 3;  
  
// Display the answer returned by  
// multiply function  
console.log("Multiplication of", x,  
            "and", y, "is", multiply(x, y));
```

String and String Functions: In Node.js we can make a variable a string by assigning a value either by using single (") or double (") quotes and it contains many functions to manipulate strings.

```
let x = "Welcome to GeeksforGeeks ";
let y = 'Node.js Tutorials';
let z = ['Geeks', 'for', 'Geeks'];
console.log(x);
console.log(y);
console.log("Concat Using (+) :", (x + y));
console.log("Concat Using Function :", (x.concat(y)));
console.log("Split string: ", x.split(' '));
console.log("Join string: ", z.join(', '));
console.log("Char At Index 5: ", x.charAt(5));
```

Output:

Welcome to GeeksforGeeks

Node.js Tutorials

Concat Using (+) : Welcome to GeeksforGeeks Node.js Tutorials

Concat Using Function : Welcome to GeeksforGeeks Node.js Tutorials

Split string: ['Welcome', 'to', 'GeeksforGeeks', " "]

Join string: Geeks, for, Geeks

Char At Index 5: m

Buffer: In node.js, we have a data type called “Buffer” to store binary data and it is useful when we are reading data from files or receiving packets over the network.

```
let b = new Buffer(10000); //creates buffer
let str = " ";
b.write(str);
console.log(str.length); //Display the information
console.log(b.length); //Display the information
```

1

10000

Node.js - REPL Terminal

The Node.js interactive REPL terminal is like the Powershell or Command prompt terminal, or a bash terminal in Linux. It performs the following tasks –

Read – Reads user's input, parses the input into JavaScript data-structure, and stores in memory.

Eval – Takes and evaluates the data structure.

Print – Prints the result.

Loop – The terminal is ready to receive next input from the user.

To start the Node.js REPL on your computer, simply enter node in the command terminal (without the javascript file name as done before). The Node.js prompt > will appear.

```
D:\nodejs>node
Welcome to Node.js v20.9.0.
Type ".help" for more information.
>
```

```
> 10+20
30
> "Hello"+"World"
'HelloWorld'
> a=10
10
> b=20
20
> a+b
30
> Math.random()
0.5423940959293392
>
```


1	.help shows the dot commands help
2	.editor enables editor mode, to write multiline JavaScript code with ease. Once you are in this mode, enter ctrl-D to run the code you wrote.
3	.break when inputting a multi-line expression, entering the .break command will abort further input. Same as pressing ctrl-C.
4	.clear resets the REPL context to an empty object and clears any multi-line expression currently being input.
5	.load loads a JavaScript file, relative to the current working directory
6	.save saves all you entered in the REPL session to a file (specify the filename)
7	.exit exits the repl (same as pressing ctrl-C two times)
8	Up/Down Keys see command history and modify previous commands.

The browser: Javascript can execute one thing at a time but that's not the case with the Browser. The Browser has its own set of APIs like setTimeout and XMLHttpRequests which are not specified in the Javascript runtime.

Heap: It's mostly the place where objects are allocated.

Callback Queue: It's a data structure that stores all the callbacks. Since it's a queue, the elements are processed based on FIFO which is First in First Out.

Event Loop: This is where all these things come together. The event loop simply checks the call stack, and if it is empty (which means there are no functions in the stack) it takes the oldest callback from the callback queue and pushes it into the call stack which eventually executes the callback.

```
console.log('hi');
```

```
setTimeout(function() {  
    console.log('freecodeCamp')  
}, 5000);
```

```
console.log('JS')
```

When the first line executes it's a `console.log()`.

This is a function invocation which means that this function is pushed into the call stack where it executes printing 'hi' to the console.

Finally it's returned and is popped off the stack.

Then when the runtime goes to execute `setTimeout()` it knows that this is a web API.

Therefore it gives it off to the browser to handle its execution.

The browser starts the timer and then JS runtime pops the `setTimeout()` out of the stack.

It encounters another `console.log()` invocation and so it pushes this into the call stack, the message 'JS' is logged into the console, and then it's finally returned.

Then the last `console.log()` is popped off the stack.

Now the call stack is empty.

Meanwhile while all of this was going on the timer finishes. When 5 seconds have elapsed the browser goes ahead and pushes the callback function into the callback queue.

Next the event loop checks if the call stack is free or not. Since it is free it takes the callback function and pushes it again back to the call stack which executes the code inside it.

Again inside the code there is a `console.log()` invocation so this function goes to the top of the stack executes which logs 'freecodecamp' into the console and finally it returns. This means it gets popped off the stack and finally the callback gets popped off the stack and we are done.

Node.js Event Loop

Node.js is a single-threaded event-driven platform that is capable of running non-blocking, asynchronous programming. These functionalities of Node.js make it memory efficient. The event loop allows Node.js to perform non-blocking I/O operations despite the fact that JavaScript is single-threaded. It is done by assigning operations to the operating system whenever and wherever possible.

Most operating systems are multi-threaded and hence can handle multiple operations executing in the background. When one of these operations is completed, the kernel tells Node.js, and the respective callback assigned to that operation is added to the event queue which will eventually be executed.

Features of Event Loop:

- An event loop is an endless loop, which waits for tasks, executes them, and then sleeps until it receives more tasks.
- The event loop executes tasks from the event queue only when the call stack is empty i.e. there is no ongoing task.
- The event loop allows us to use callbacks and promises.
- The event loop executes the tasks starting from the oldest first.

```
console.log("This is the first statement");
```

```
setTimeout(function(){  
    console.log("This is the second statement");  
}, 1000);
```

```
console.log("This is the third statement");
```


Output:

This is the first statement

This is the third statement

This is the second statement

Explanation:

In the above example, the first console log statement is pushed to the call stack, and “This is the first statement” is logged on the console, and the task is popped from the stack.

Next, the `setTimeout` is pushed to the queue and the task is sent to the Operating system and the timer is set for the task.

This task is then popped from the stack. Next, the third console log statement is pushed to the call stack, and “This is the third statement” is logged on the console and the task is popped from the stack.

When the timer set by the `setTimeout` function (in this case 1000 ms) runs out, the callback is sent to the event queue.

The event loop on finding the call stack empty takes the task at the top of the event queue and sends it to the call stack.

The callback function for the `setTimeout` function runs the instruction and “This is the second statement” is logged on the console and the task is popped from the stack.

Working of the Event loop: When Node.js starts, it initializes the event loop, processes the provided input script which may make async API calls, schedules timers, then begins processing the event loop. In the previous example, the initial input script consisted of `console.log()` statements and a `setTimeout()` function which schedules a timer.

When using Node.js, a special library module called `libuv` is used to perform async operations.

This library is also used, together with the back logic of Node, to manage a special thread pool called the `libuv` thread pool.

This thread pool is composed of four threads used to delegate operations that are too heavy for the event loop. I/O operations, Opening and closing connections, `setTimeouts` are examples of such operations.

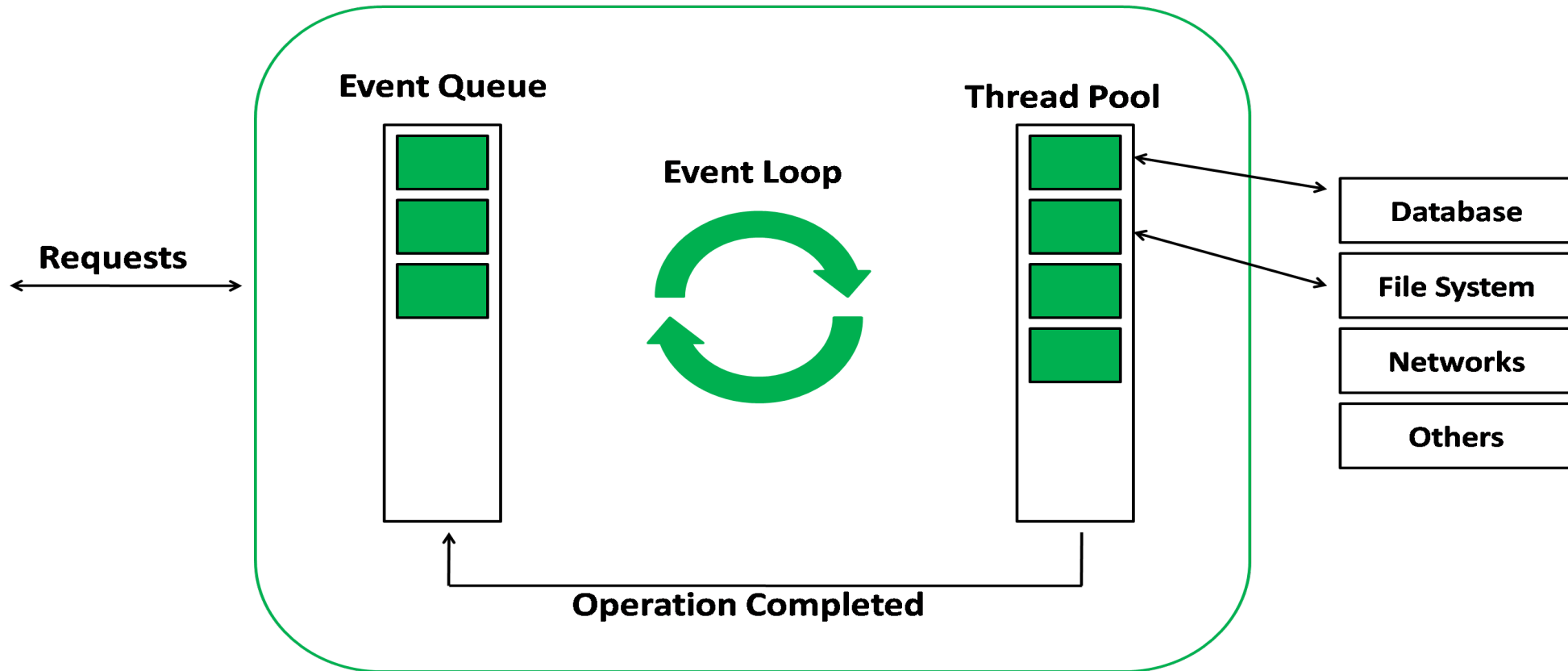
When the thread pool completes a task, a callback function is called which handles the error(if any) or does some other operation.

This callback function is sent to the event queue.

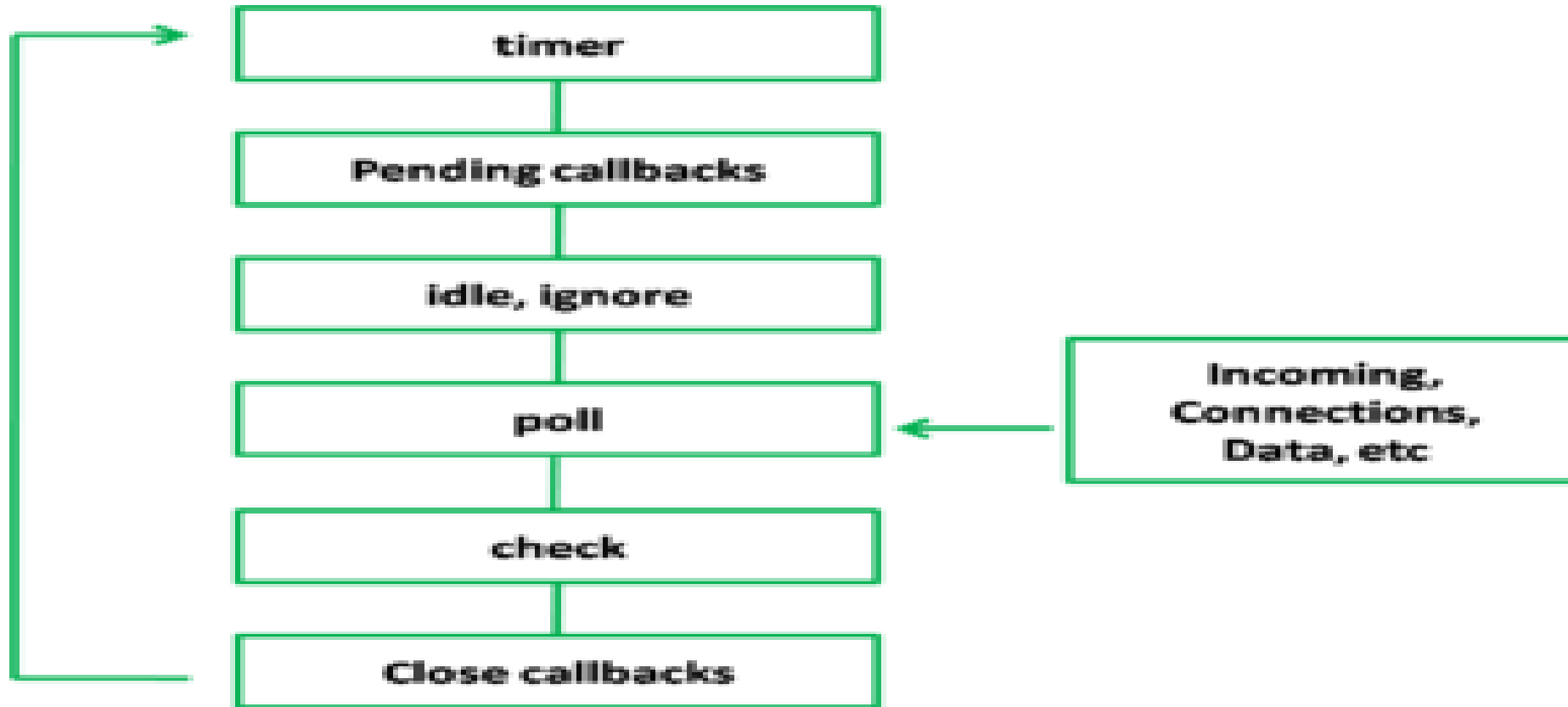
When the call stack is empty, the event goes through the event queue and sends the callback to the call stack.

The following diagram is a proper representation of the event loop in a Node.js server:

Node.js Server



Phases of the Event loop: The event loop in Node.js consists of several phases, each of which performs a specific task. These phases include:



1. Timers: This phase processes timers that have been set using `setTimeout()` and `setInterval()`.

```
console.log('Start');
```

```
setTimeout(() => {
```

```
    console.log('Timeout callback');
```

```
}, 2000);
```

```
console.log('End');
```

In this example, the `setTimeout()` function is called with a callback that will print “Timeout callback” to the console after 2000 milliseconds (2 seconds). This function is added to the message queue in the timers phase, and the event loop will process it after the synchronous code is executed.

The output will be:

Start

End

Timeout callback

As you can see, the “Timeout callback” is printed after 2 seconds, after the “Start” and “End” are printed, because the `setTimeout()` function is non-blocking and its callback is processed by the event loop after the execution of the synchronous code.

2. Pending Callbacks: This phase processes any callbacks that have been added to the message queue by asynchronous functions.

```
console.log('Start');  
setImmediate(() => {  
    console.log('Immediate callback');  
});  
console.log('End');
```

In this example, the `setImmediate()` function is called with a callback that will print “Immediate callback” to the console.

This function is added to the message queue in the pending callbacks phase, and the event loop will process it after the timers phase.

The output will be:

Start

End

Immediate callback

3. Idle, Prepare: The “idle.ignore” phase is not a standard phase of the event loop in Node.js.

It means it's Used internally only.

The “idle” phase is a period of time during which the event loop has nothing to do and can be used to perform background tasks, such as running garbage collection or checking for low-priority events.

```
const { idle } = require('idle-gc');
```

```
idle.ignore();
```

4. Poll: This phase is used to check for new I/O events and process any that have been detected.

```
const fs = require('fs');  
const readStream = fs.createReadStream('./file.txt');  
console.log('Start');  
readStream.on('data', (chunk) => {  
    console.log(chunk.toString());  
});  
console.log('End');
```

In this example, a `readStream` is created to read the contents of a file. The 'data' event is added to the message queue in the poll phase, and the event loop will process it after the pending callbacks phase. The output will be the content of the file.

5. Check This phase processes any `setImmediate()` callbacks that have been added to the message queue.

```
console.log('Start');  
setImmediate(() => {  
    console.log('Immediate callback');  
});  
console.log('End');
```

In this example, the `setImmediate()` function is called with a callback that will print “Immediate callback” to the console.

This function is added to the message queue in the check phase, and the event loop will process it after the poll phase.

The output will be:

Start

End

Immediate callback

6. Close Callbacks: This phase processes any callbacks that have been added to the message queue by the close event of a socket. This means that any code that needs to be executed when a socket is closed is placed in the message queue and processed during this phase.

```
const net = require('net');

const server = net.createServer((socket) => {
  socket.on('close', () => {
    console.log('Socket closed');
  });
});

server.listen(8000);
```

In this example, a server is created using the net module, and the 'close' event is added to the message queue in the close callbacks phase with a callback that will print "Socket closed" to the console.

This event will be triggered when the server's socket is closed by the client and the callback will be processed by the event loop during the close callbacks phase.

It's important to note that the order of execution of these phases can vary depending on the specific implementation of the event loop, but generally, the event loop will process them in the order mentioned above.

Each phase is executed in order, and the event loop will continue to cycle through these phases until the message queue is empty.

Conclusion:

The event loop is a powerful feature of Node.js that enables it to handle a high number of concurrent connections and perform non-blocking I/O operations.

Understanding how the event loop works is essential for building efficient and performant server-side applications in Node.js.

With a better understanding of the event loop, developers can take full advantage of the capabilities of Node.js and build high-performance, scalable applications.

Node Callback

A callback in Node is a non-blocking function that executes upon task completion, enabling asynchronous processing. It facilitates scalability by allowing Nodejs to handle multiple requests without waiting for operations to conclude.

Building Http Server

Creating Web Servers Using NodeJS: There are mainly two ways as follows.

1. Using http inbuilt module
2. Using express third party module

Using http module: HTTP and HTTPS, these two inbuilt modules are used to create a simple server.

The HTTPS module provides the feature of the encryption of communication with the help of the secure layer feature of this module.

Whereas the HTTP module doesn't provide the encryption of the data.

Project structure: It will look like this.

```
▼ NODE.JS
  JS index.js
  {} package.json
```

```
// Importing the http module  
const http = require("http")
```

```
// Creating server  
const server = http.createServer((req, res) => {  
    // Sending the response  
    res.write("This is the response from the server")  
    res.end();  
})
```

```
// Server listening to port 3000  
server.listen((3000), () => {  
    console.log("Server is Running");  
})
```

Run index.js file using below command:node index.js

```
lenovo@LAPTOP-OBEPNKMU MINGW64 ~/Desktop/Node.js  
$ node index.js  
Server is Running
```

Now open your browser and go to ***http://localhost:3000/***, you will see the following output:

A screenshot of a web browser's address bar. It features navigation icons (back, forward, refresh) on the left, an information icon, and the text 'localhost:3000' in the center.

← → ↻ ⓘ localhost:3000

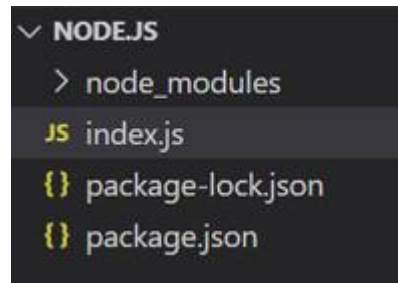
This is the response from the server

Using express module: The express.js is one of the most powerful frameworks of the node.js that works on the upper layer of the http module. The main advantage of using express.js server is filtering the incoming requests by clients.

Installing module: Install the required module using the following command.

`npm install express`

Project structure: It will look like this.



```
// Importing express module
const express = require("express")
const app = express()
```

```
// Handling GET / request
app.use("/", (req, res, next) => {
    res.send("This is the express server")
})

// Handling GET /hello request
app.get("/hello", (req, res, next) => {
    res.send("This is the hello response");
})

// Server setup
app.listen(3000, () => {
    console.log("Server is Running")
})
```


Run **the index.js** file using the below command:

```
node index.js
```

Output: Now open your browser and go to ***http://localhost:3000/***, you will see the following output:



A screenshot of a web browser's address bar. It features navigation icons (back, forward, refresh) on the left, followed by an information icon and the text 'localhost:3000'.

This is the express server

`app.get()` is a function that tells the server what to do when a get request at the given route is called.

The `app.use()` function is used to mount the specified middleware function(s) at the path that is being specified.

`app.get` is called when the HTTP method is set to GET , whereas `app.use` is called regardless of the HTTP method, and therefore defines a layer which is on top of all the other RESTful types which the express packages gives you access to

Import and Export in Node.js

In programming, modules are components of a program with one or more functions or values.

These values can also be shared across the entire program and can be used in different ways.

Why Export Modules?

we use them in other parts of your application.

Modules can serve different purposes. They can provide simple utilities to modify strings. They can provide methods for making API requests. Or they can even provide constants and primitive values.

When you export a module, you can import it into other parts of your applications and consume it.

Node.js supports CommonJS Modules and ECMAScript Modules.

How to Export Modules in Node

Node.js already exports in-built modules which include fs, path, and http to name a few. But you can create your own modules.

Node.js treats each file in a Node project as a module that can export values and functions from the file.

module.exports vs exports in Node

You can export functions and values from a module by either using `module.exports`:

```
module.exports = { value1, function1 }
```

or by using `exports`:

```
exports.value1 = value1
```

```
exports.function1 = function1
```

These methods are pretty identical. Basically, `exports` serves as a reference to `module.exports`.

Each file in a Node.js project is treated as a module that can export values to be used by other modules.

`module.exports` is an object in a Node.js file that holds the exported values and functions from that module.

Declaring a `module.exports` object in a file specifies the values to be exported from that file. When exported, another module can import this values with the `require` global method.

Steps to include functions from other files:

1.Creating a Module: Modules are created in Node.js are JavaScript files. Every time a new file with **.js** extension is created, it becomes a module.

1.Exporting a Module: Filename: func.js

```
function add(x, y) {  
  return x + y;  
}
```

```
function subtract(x, y) {  
  return x - y;  
}
```

```
// Adding the code below to allow importing
```

```
// the functions in other files
```

```
module.exports = { add }
```

Importing a Module: We need to import the module to use the functions defined in the imported module in another file. The result returned by require() is stored in a variable which is used to invoke the functions using the dot notation. **Filename: main.js**

```
// Importing the func.js module
```

```
// The ./ says that the func module is in the same directory as the main.js file
```

```
// const f = require('./func');
```

```
// Require returns an object with add() and stores it in the f variable which is used to invoke the required
```

```
const result = f.add(10, 5);
```

```
console.log('The result is:', result);
```


Importing multiple functions from local file: Filename: func.js

```
function add(x, y) {  
    return x + y;  
}  
  
function subtract(x, y) {  
    return x - y;  
}  
  
module.exports = { add, subtract};
```

Filename: main.js

```
const f = require('./func');  
  
console.log(f.add(4, 4));  
console.log(f.subtract(8, 4));
```

There are three types of modules in Node.js

Importing from local module:

These modules are created by the user and can be imported as:

```
const var = require('./filename.js'); // OR
```

```
const var = require('./path/filename.js');
```

Importing from core modules:

These modules are inbuilt in Node.js and can be imported as:

```
const var = require('fs');
```

Importing from third party modules:

These modules are installed using a package manager such as npm.

Examples of third party modules are express, mongoose, nodemon, etc.

These are imported as:

```
const express = require('express');
```

When to use Node.js?

- Node.js is good for creating streaming based real- time services, web chat applications, static file servers etc.
- If you need high level concurrency and not worried about CPU-cycles.
- If you are great at writing JavaScript code because then you can use the same language at both the places: server-side and client-side.

When not to use Node.js?

- When you are doing heavy and CPU intensive calculations on server side, because event-loops are CPU hungry.
- Node's relational database support tools are not up to the expected level when compared to other languages.